

Chapter 11 Constructors & Destructors

Constructors: -

A constructor is a special member function which is called automatically just after the object is created. It is the first function called by the object. It is special because its name is same as the class name. It can be used to initialize the instance member variables of the class. It can be used to open a file or any other resource. It can be used to provide dynamic memory to its members. It is called constructor because it constructs the values of data members of the class.

- It should be defined in public block.
- It does not have any return type.
- If we don't define any constructor, then compiler will add a default constructor in the class.
- A class can have more than one constructors.
- Constructor cannot be inherited. (Covered in Inheritance Chapter)
- Constructor cannot be virtual. (Covered in Virtual Function Chapter)
- Constructor can be used as a conversion function. (Covered in Operator Overloading Chapter)

Destructors: -

Destructor is an instance member function which is invoked automatically whenever an object is going to be destroyed. Meaning, a destructor is the last function that is going to be called before an object is destroyed. The name of a destructor is also same as of class name but a tilde ~ symbol is preceded. It is called automatically when an object goes out of scope. It can be used for clean up code such as to free any dynamic memory or to close a file etc.

- Destructor must be defined in public section.
- They do not have return types, not even void and therefore, they cannot return values.
- Every class contains a default destructor if no explicit destructor is defined.
- A class can't have more than one destructor because it cannot accept any argument. So destructor overloading is not possible.
- They cannot be inherited.
- They can be virtual.
- It cannot be declared static or const.

When do we need to write a user-defined destructor?

If we do not write our own destructor in class, compiler creates a default destructor for us. The default destructor works fine unless we have dynamically allocated memory or pointer in class. When a class contains a pointer to memory allocated in class, we should write a destructor to release memory before the class instance is destroyed. This must be done to avoid memory leak.

Matrix Computers (Page 2 of 14)

10/564 Kaveri Path, Mansarovar Jaipur-302020 Rajasthan
9414752750, 9414930104, 9414244351, 0141-2399633

We Provide online/offline training in



Special classes to solve questions of [hackerrank.com](https://www.hackerrank.com), [hackerearth.com](https://www.hackerearth.com), [codechef.com](https://www.codechef.com)

Free
Demo
Classes

Repeat
Option
Available

Free
Study
Material

We provide internship, summer training for B. Tech students

Example: -

```
#include<iostream>
using namespace std;
class A{
    int x,y;
public:
    A(){//default zero argument constructor
        x=y=0;
    }
    A(int x1){//parameterized one argument constructor
        x=y=x1;
    }
    A(int x1, int y1){//parameterized two argument constructor
        x=x1;
        y=y1;
    }
    void getdata(){
        cout<<"Enter 2 numbers ";
```

Matrix Computers (Page 3 of 14)

10/564 Kaveri Path, Mansarovar Jaipur-302020 Rajasthan

9414752750, 9414930104, 9414244351, 0141-2399633

```
        cin>>x>>y;
    }
    void display(){
        cout<<x<<"\t"<<y<<endl;
    }
    ~A(){ //destructor
    }
};
int main(void){
    A a1;
    a1.display();
    A a2(5); //A a2=5;implicit conversion
    a2.display();
    A a3=A(7,8);
    a3.display();
    return 0;
}
```

Type of Constructors: -

1. A constructor that accepts no parameters is called the **default constructor**.
2. A constructor that accepts arguments is called as **parameterized constructors**.
3. A constructor that accepts default arguments is called as **default argument constructor**.
4. A constructor allocating dynamic memory using new operator is called as **dynamic constructor**.
5. A constructor used to declare and initialize an object from another object is called as **copy constructor**.

1. Default Constructor: -

Default constructor is the constructor which doesn't take any argument. It has no parameters. Even if we do not define any constructor explicitly, the compiler will automatically provide a default constructor implicitly.

Example of default constructor defined by us: -

```
#include<iostream>
class A{
    int x,y;
    public:
    A(){
        x=y=0;
    }
    void display(){
        cout<<x<<" "<<y<<endl;
    }
}
```

Matrix Computers (Page 4 of 14)

10/564 Kaveri Path, Mansarovar Jaipur-302020 Rajasthan

9414752750, 9414930104, 9414244351, 0141-2399633

```
};  
int main(){  
    A a1;  
    a1.display();        //0,0  
    return 0;  
}
```

Example of default constructor inserted implicitly by the compiler: -

```
#include<iostream>  
class A{  
    int x,y;  
    public:  
    void display(){  
        cout<<x<<" "<<y<<endl;  
    }  
};  
int main(){  
    A a1;  
    a1.display();        //garbage,garbage  
    return 0;  
}
```

2. Parameterized Constructor: -

A constructor that accepts one or more arguments is called as parameterized constructor. These arguments help to initialize the members of an object when it is created. When an object is declared in a parameterized constructor, the initial values have to be passed as arguments to the constructor function.

Example: -

```
class A{  
    int x,y;  
    public:  
    A(int x1, int y1){  
        x=x1;  
        y=y1;  
    }  
    void display(){  
        cout<<x<<" "<<y<<endl;  
    }  
};  
int main(){  
    A a1(5,6);  
    a1.display();        //5,6  
    return 0;  
}
```

```
}
```

Compiler will not add default constructor if parameterized constructor is defined by us: -

Compiler will not add default zero argument constructor if any explicit parameterized constructor is defined by us. So if we create any object without passing any argument then there will be error. So we have to pass arguments. So it is best practice to always define a default constructor.

Example: -

```
class A{
    int x,y;
    public:
    A(int x1, int y1){
        x=x1;
        y=y1;
    }
    void display(){
        cout<<x<<" "<<y<<endl;
    }
};
int main(){
    A a1;                //error as no default constructor is available.
    A a[5];              //error as it will call default constructor 5 times
    A a2(5,6);           //It will work
    return 0;
}
```

Constructor can be called Explicitly or Implicitly: -

A a1(5,6); Implicitly called as constructor name is missing.

A a1 = A(5,6); Explicitly called as constructor name is present.

No return type: -

They do not have return types, not even void because they cannot return values.

Example: -

```
class A{
    public:
    int A(){                //Error as we can't add return type in constructor
        return 5;
    }
    void A(int n){          //Error as it is implicitly void
    }
```

```
    }  
};
```

Constructor Overloading: -

A class can have more than one constructors if their arguments are different. This is called as constructor overloading.

Example: -

```
class A{  
    int x,y;  
    public:  
    A(){  
        x=y=0;  
    }  
    A(int n){  
        x=y=n;  
    }  
    A(int x, int y){  
        this->x=x;  
        this->y=y;  
    }  
    void display(){  
        cout<<x<<" "<<y<<endl;  
    }  
};  
int main(){  
    A a1;  
    a1.display();  
    A a2(5);  
    a2.display();  
    A a3(5,6);  
    a3.display();  
    return 0;  
}
```

Constructors should be defined in public section: -

We should define constructor in public section because it is called outside the class. We can also define constructor as private but then we have to create all objects inside the member functions not outside.

Example: -

```
class A{  
    int x,y;
```

Matrix Computers (Page 7 of 14)

10/564 Kaveri Path, Mansarovar Jaipur-302020 Rajasthan

9414752750, 9414930104, 9414244351, 0141-2399633

```
A(int x1, int y1){           //Constructor is private
    x=x1;
    y=y1;
}
public:
void display(){
    cout<<x<<" "<<y<<endl;
}
};
int main(){
    A a1(5,6);               //error as constructor is private and calling outside
    return 0;
}
```

Example: -(static will be covered in next chapter)

```
class A{
    int x;
    A(int x){                 //Constructor is private
        this->x=x;
        cout<<"Constructor called"<<endl;
    }
    public:
    static A getobj(int n){
        return A(n);         //Constructor called inside member function so
        no error
    }
    void display(){
        cout<<x<<endl;
    }
};
int main(){
    A a1=A::getobj(5);
    a1.display();
    A a2=A::getobj(6);
    a2.display();
    return 0;
}
```

3. Default Argument Constructor: -

A constructor in which default values are provided in their arguments is called as Default Argument Constructor. We have to follow all rules of Default Arguments in this case.

Example of Default Argument Constructor: -

Matrix Computers (Page 8 of 14)

10/564 Kaveri Path, Mansarovar Jaipur-302020 Rajasthan

9414752750, 9414930104, 9414244351, 0141-2399633

```
class A{
    int x,y;
    public:
    A(int x=0, int y=0){           //Default Argument Constructor
        this->x=x;
        this->y=y;
    }
    void display(){
        cout<<x<<" "<<y<<endl;
    }
};
int main(){
    A a1;
    a1.display(); //0,0
    A a2(5);
    a2.display(); //5,0
    A a3(5,6);
    a3.display(); //5,6
    return 0;
}
```

Note: - If Default argument constructor is defined then we should remove all other constructors otherwise there may be ambiguity in calling constructors.

Example : -

```
class A{
    int x,y;
    public:
    A(){
        x=y=0;
    }
    A(int n){
        x=y=n;
    }
    A(int x=0, int y=0){ //Default Argument Constructor
        this->x=x;
        this->y=y;
    }
    void display(){
        cout<<x<<" "<<y<<endl;
    }
};
int main(){
    A a1;           //Ambiguity error as compiler is confused in two constructors
```


Matrix Computers (Page 9 of 14)

10/564 Kaveri Path, Mansarovar Jaipur-302020 Rajasthan

9414752750, 9414930104, 9414244351, 0141-2399633

```
a1.display();
A a2(5);           //Ambiguity error as compiler is confused in two constructors
a2.display();
A a3(5,6);        //No error
a3.display();
return 0;
}
```

4. Dynamic Constructor: -

If we create any dynamic memory using new in any constructor then that constructor is called as dynamic constructor.

Example: - This example shows the syntax of dynamic constructor. Here a1 object is created with argument x, one parameter constructor will be called in this case and new int[x] will allocate memory for x integer elements. Note that the memory block which we allocate dynamically through "new" must be deleted (free) manually before terminating the program, the compiler will not free the memory by its own.

```
#include<iostream>
class A{
    int *p;
    int n;
public:
    A(){
        n=0;
        p=NULL;
    }
    A(int n){           //Dynamic Constructor
        this->n=n;
        p=new int [n];
    }
    void getdata();
    void display();
    ~A(){               //Destructor
        delete []p;
    }
};
void A::getdata(){
    for(int i=0;i<n;i++){
        cout<<"Enter number: "<<i+1<<";
        cin>>p[i];
    }
}
void A::display(){
```

```
        for(int i=0;i<n;i++)
            cout<<p[i]<<"\t";
    }
    int main (void){
        int x;
        cout<<"Enter number of elements: ";
        cin>>x;
        A a1(x); // or A a1=x
        a1.getdata();
        a1.display();
        return 0;
    }
```

5. Copy Constructor: -

A Constructor which copies the value of all the member variables of one object into another object is called as Copy Constructor. A copy constructor is a member function that initializes an object using another object of the same class. A copy constructor has the following syntax:

```
ClassName (const ClassName &old_obj);
```

When is copy constructor called?

1. When an object of the class is returned by value.
2. When an object of the class is passed (to a function) by value as an argument.
3. When an object is constructed based on another object of the same class.
4. When the compiler generates a temporary object.

Note: - It is, however, not guaranteed that a copy constructor will be called in all these cases, because the C++ Standard allows the compiler to optimize the copy away in certain cases, one example is the return value optimization (sometimes referred to as RVO).

When is a user-defined copy constructor needed?

If we don't define our own copy constructor, the C++ compiler creates a default copy constructor for each class which does a member-wise copy between objects. The compiler created copy constructor works fine in general. We need to define our own copy constructor only if an object has pointers or any runtime allocation of the resource like file-handle, a network connection etc.

The default constructor does only shallow copy.

Deep copy is possible only with user defined copy constructor. In user defined copy constructor, we make sure that pointers (or references) of copied object point to new memory locations.

Example: -

//copy constructor (shallow Copy)(Default Copy Constructor)

```
#include<iostream>
```

```
using namespace std;
```

```
class A{
```

```
    public:
```

```
        int *p;
```

```
        int n;
```

```
        A(int n){
```

```
            this->n=n;
```

```
            p=new int[n];
```

```
        }
```

```
        void display(){
```

```
            for(int i=0;i<n;i++)
```

```
                cout<<p[i]<<"\t";
```

```
            cout<<endl;
```

```
        }
```

```
        ~A(){
```

```
            delete [] p;
```

```
        }
```

```
};
```

```
int main(void){
```

```
    A a1(3);
```

```
    a1.p[0]=10;
```

```
    a1.p[1]=20;
```

```
    a1.p[2]=30;
```

```
    A a2(a1);
```

```
    //copy constructor
```

```
    a2.p[0]=11;
```

```
    //a1 will also change
```

```
    a1.display();
```

```
    a2.display();
```

```
    return 0;
```

```
}
```

Example: -

//copy constructor (Deep Copy)(Explicit Copy Constructor)

```
#include<iostream>
```

```
using namespace std;
```

```
class A{
```

```
    public:
```

```
        int *p;
```

```
        int n;
```

```
        A(){
```

Matrix Computers (Page 12 of 14)

10/564 Kaveri Path, Mansarovar Jaipur-302020 Rajasthan

9414752750, 9414930104, 9414244351, 0141-2399633

```
        p=NULL;
        n=0;
    }
    A(int n){
        this->n=n;
        p=new int[n];
    }
    A(const A &r){
        n=r.n;
        p=new int[n];
        for(int i=0;i<n;i++)
            p[i]=r.p[i];
    }
    void display(){
        for(int i=0;i<n;i++)
            cout<<p[i]<<"\t";
        cout<<endl;
    }
    ~A(){
        delete [] p;
    }
};

int main(void){
    A a1(3);
    a1.p[0]=10;
    a1.p[1]=20;
    a1.p[2]=30;

    A a2(a1);
    a2.p[0]=11;           //a1 will not change

    a1.display();
    a2.display();
    return 0;
}
```

Copy constructor vs Assignment Operator: -

Which of the following statements call copy constructor and which one calls assignment operator?

```
A a1(5,6);
A a2(a1);      (1)
A a3=a1;      (1)
A a4;
```

a4=a1; (2)

Copy constructor is called when a new object is created from an existing object, as a copy of the existing object. Assignment operator is called when an already initialized object is assigned a new value from another existing object. In the above example (1) calls copy constructor and (2) calls assignment operator.

Can we make copy constructor private?

Yes, a copy constructor can be made private. When we make a copy constructor private in a class, objects of that class become non-copyable. This is particularly useful when our class has pointers or dynamically allocated resources. In such situations, we can either write our own copy constructor or make a private copy constructor so that users get compiler errors rather than surprises at runtime.

Why argument to a copy constructor must be passed as a reference?

A copy constructor is called when an object is passed by value. Copy constructor itself is a function. So if we pass an argument by value in a copy constructor, a call to copy constructor would be made to call copy constructor which becomes a non-terminating chain of calls. Therefore compiler doesn't allow parameters to be passed by value.

Why argument to a copy constructor should be const?

Because argument of copy constructor is passed by reference so copy constructor can change this argument. If we don't want our object to be changed then we should use const keyword.

Equal Operator Overloading: -

Like constructors or destructor every class has its own default overloaded equal to operator(=) member function, that is used to copy one object data members to another object data members so if we still don't have our own overloaded equal to operator member function the statement "a3=a2=a1" will still work fine. Like constructors if we don't define our own then compiler will define it but this will also shallow copy. If we have a pointer as a member inside any object then we have to define our own equal operator.

Example: -

```
//copy constructor (Deep Copy)(Explicit Copy Constructor)
#include<iostream>
using namespace std;
class A{
public:
    int *p;
    int n;
    A(){
        p=NULL;
```

Matrix Computers (Page 14 of 14)

10/564 Kaveri Path, Mansarovar Jaipur-302020 Rajasthan

9414752750, 9414930104, 9414244351, 0141-2399633

```
        n=0;
    }
    A(int n){
        this->n=n;
        p=new int[n];
    }
    A operator=(A obj){
        n=obj.n;
        p=new int[n];
        for(int i=0;i<n;i++)
            p[i]=obj.p[i];
    }
    void display(){
        for(int i=0;i<n;i++)
            cout<<p[i]<<"\t";
        cout<<endl;
    }
    ~A(){
        delete [] p;
    }
};
int main(void){
    A a1(3),a2;
    a1.p[0]=10;
    a1.p[1]=20;
    a1.p[2]=30;

    a2=a1;
    a2.p[0]=11;           //a1 will not change

    a1.display();
    a2.display();
    return 0;
}
```